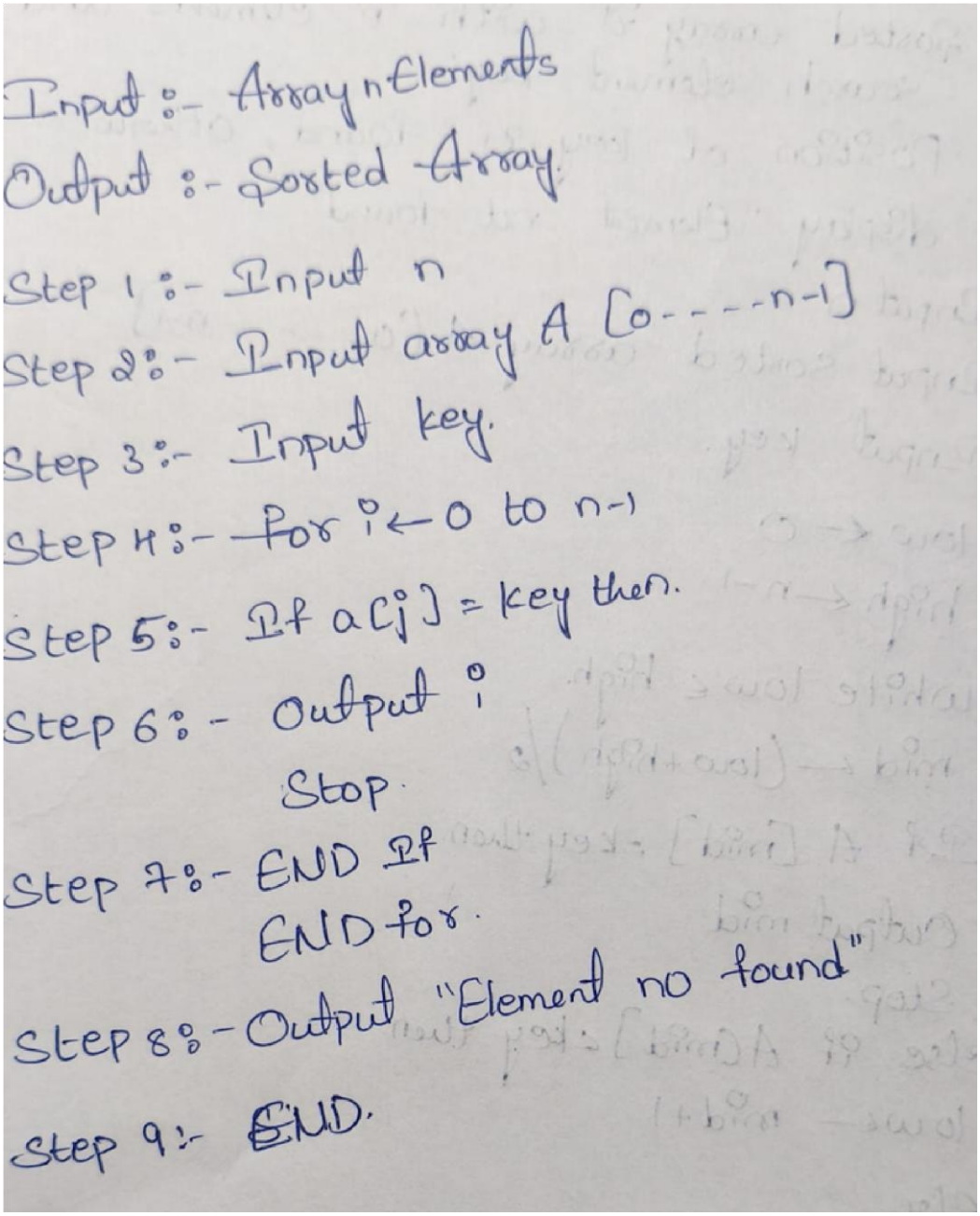
Practical-2

Aim:

Implementation and Time analysis of linear and binary search algorithm.

1.Linear search Algorithm:



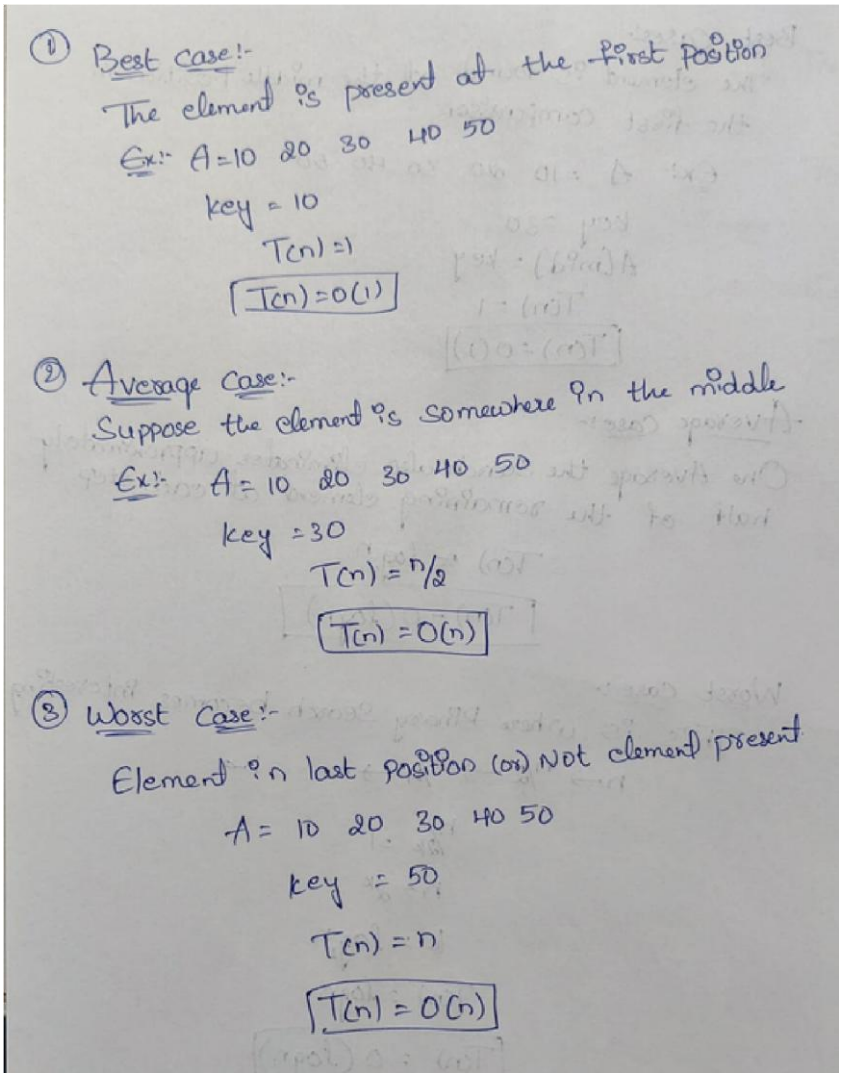
Program code:

```
#include <iostream>
using namespace std;

int linearsearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;  // return index if found
        }
    }
    return -1;  // return -1 if not found
}
int main() {
    int arr[] = {2, 3, 4, 5, 6, 7, 8};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 8;
    int result = linearsearch(arr, n, key);

    if (result == -1) {
        cout << "Element not found";
    } else {
        cout << "The element found at index " << result;
    }
    return 0;
}
```

Output:

```
The element found at index 6
```

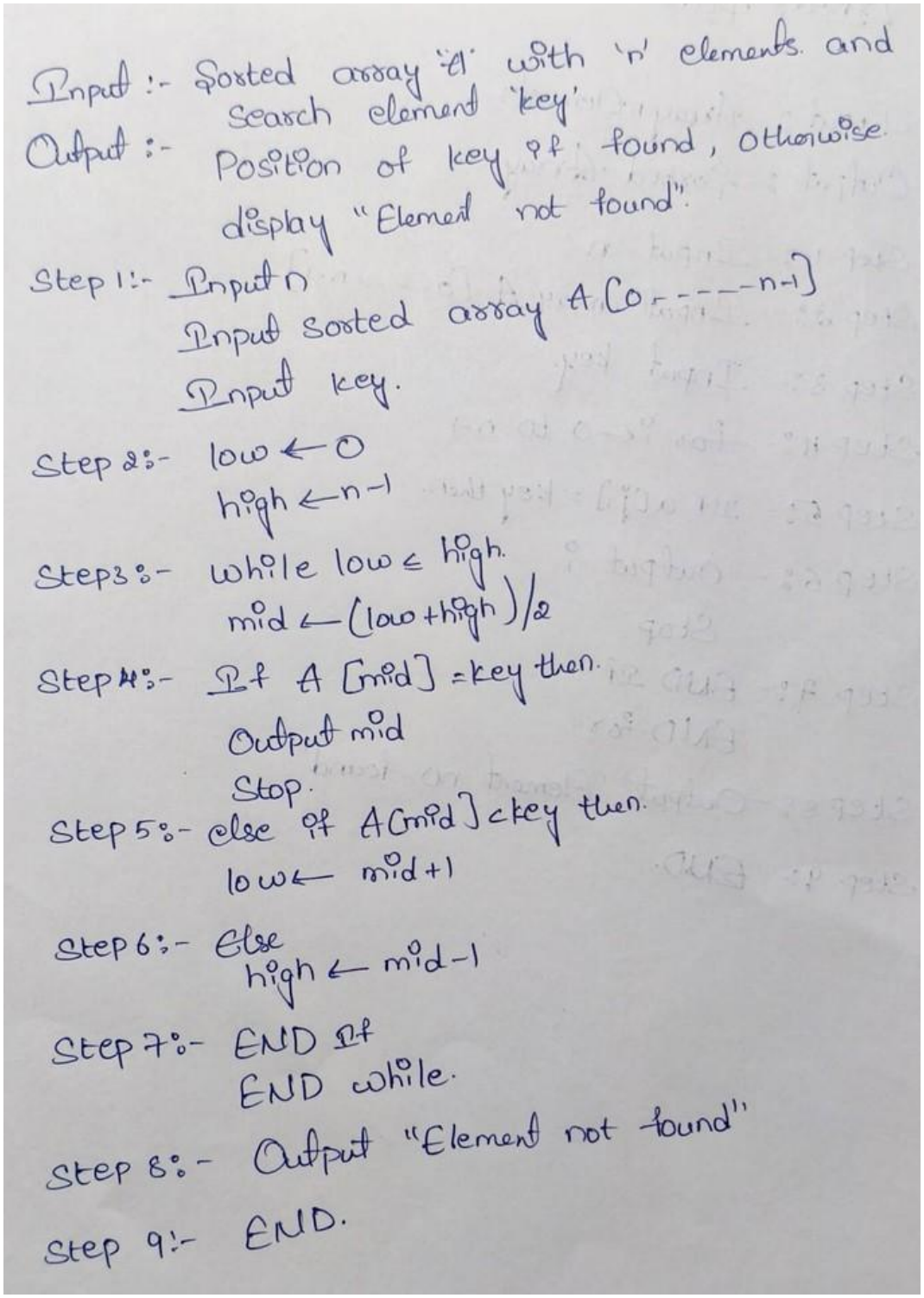
Time complexity:



Conclusion:

Linear search searches the array by comparing each element with the key one by one. It returns the position of the key if it is found; otherwise, it reports that the element is not found.

2.Binary search **Algorithm:**



Program:

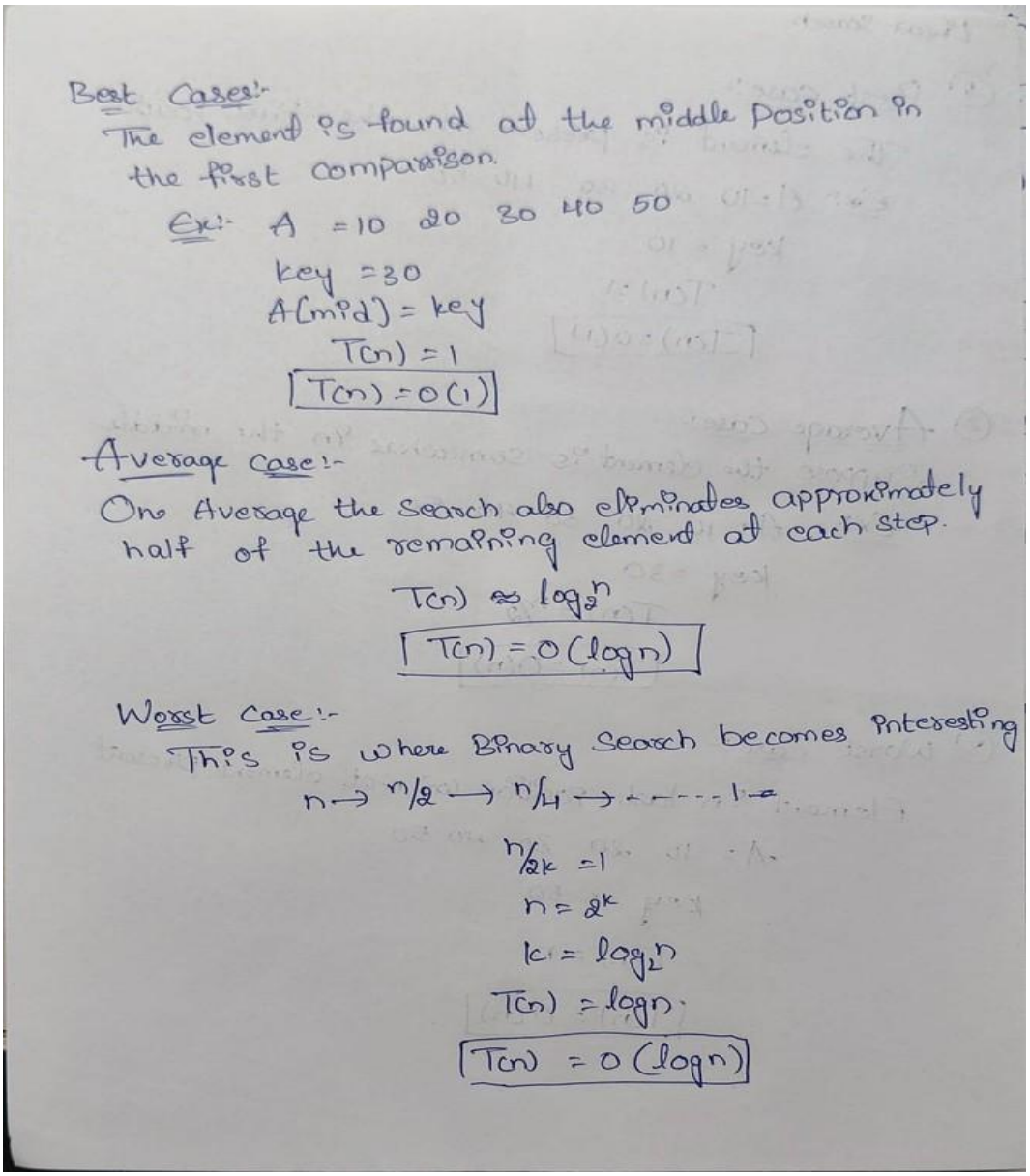
```
#include <iostream>
using namespace std;

int binarysearch(int arr[], int size, int key) {
   int low = 0;
   int high = size - 1;

   while (low <= high) {
      int mid = (low + high) / 2;

      if (arr[mid] == key) {
         return mid; // found
      }
      else if (arr[mid] < key) {
         low = mid + 1; // search right half
      }
      else {
         high = mid - 1; // search left half
      }
   }
   return -1; // not found
}
```

```
int main() {
    int arr[] = {1, 2, 3, 4, 5, 6};
    int size = sizeof(arr) / sizeof(arr[0]);
    int key = 5;

    int result = binarysearch(arr, size, key);

    if (result == -1) {
        cout << "The element not found";
    } else {
        cout << "The element found at the index " << result;
    }

    return 0;
}
```

Output:

```
The element found at the index 4
```

Time complexity:



Conclusion:

Binary search searches the sorted array by repeatedly dividing the search range into two halves. It efficiently finds the key element and returns its position if found.